Python Strings: Text Made Easy

Slice, change and search text like a pro.

Python | Beginner | Ages 9–16

InfiBee Coders

The Big Idea

- A string is text: letters, numbers and symbols inside quotes.
- You can use single `'...'` or double `"..."` quotes. Just match them.
- Each character has a position called an index, starting at 0.
- Strings cannot be changed in place. String methods give you a new string.
- Strings are used everywhere: names, messages, passwords and web pages.

Indexing and Slicing

- `word[0]` is the first character and `word[-1]` is the last.
- `word[1:4]` takes characters 1, 2 and 3. The stop index is not included.
- `word[:3]` is the first three, `word[3:]` is everything from index 3.
- `word[::-1]` reverses the string.
- `len(word)` counts the characters.

Useful String Methods

- `.upper()`, `.lower()`, `.title()` change the case.
- `.strip()` removes spaces from both ends.
- `.replace("a", "b")` swaps text.
- `.split()` breaks a string into a list. `" ".join(list)` joins a list back.
- `.find(x)` gives the position (or -1). `.count(x)` counts matches.
- `.startswith(x)` and `.endswith(x)` return True or False.

Formatting and Special Characters

- f-strings: `f"Hello {name}"` is the easiest way to mix text and values.
- `\n` is a new line and `\t` is a tab.
- Triple quotes `"""..."""` let a string run across many lines.
- `in` checks if text is inside a string: `"cat" in "concatenate"`.

💻 Code Examples

Index and slice — python

```python
word = "PYTHON"
print(word[0])
print(word[-1])
print(word[1:4])
print(word[:2])
print(word[::-1])
print(len(word))
```

▶ **Output:**

```
P
N
YTH
PY
NOHTYP
6
```

Index 0 is P, -1 is N, 1:4 is YTH.

Change the case — python

```python
name = "mia sharma"
print(name.upper())
print(name.title())
print("  hello  ".strip())
```

▶ **Output:**

```
MIA SHARMA
Mia Sharma
hello
```

The original string is not changed. Each method makes a new one.

Replace, find and count — python

```python
text = "I like cats. Cats are great."
print(text.replace("cats", "dogs"))
print(text.lower().count("cats"))
print(text.find("like"))
```

▶ **Output:**

```
I like dogs. Cats are great.
2
2
```

lower() first so Cats and cats both match.

Split and join python

```python
sentence = "learn python every day"
words = sentence.split()
print(words)
print(len(words))
print("-".join(words))
```

▶ **Output:**

```
['learn', 'python', 'every', 'day']
4
learn-python-every-day
```

split() makes a list of words. join() glues them back with a separator.

f-strings with calculations python

```python
item = "pencil"
price = 12.5
qty = 4
print(f"{qty} x {item} = Rs {price * qty}")
print(f"{price:.2f}")
```

▶ **Output:**

```
4 x pencil = Rs 50.0
12.50
```

The :.2f part shows 2 decimal places.

Loop through a string python

```python
for letter in "code":
    print(letter.upper())
vowels = 0
for ch in "education":
    if ch in "aeiou":
        vowels = vowels + 1
print("Vowels:", vowels)
```

▶ **Output:**

```
C
O
D
E
Vowels: 5
```

A string works like a list of characters.

📚 Key Terms

String
Text inside quotes.

Index
The position of a character, starting at 0.

Slice
A part of a string, like word[1:4].

Method
A function that belongs to a value, like .upper().

Concatenation
Joining strings with +.

f-string
A string with values inside {}.

Immutable
Cannot be changed in place.

Escape character
A backslash that gives special meaning, like \n.

len()
The number of characters.

Substring
A smaller piece of a string.

🧮 Quick Reference

```
s[0]  s[-1]  s[1:4]  s[::-1]
len(s)
s.upper()  s.lower()  s.title()
s.strip()  s.replace("a", "b")
s.split()  "-".join(items)
s.find("x")  s.count("x")
s.startswith("a")  s.endswith("z")
f"Hello {name}"
```

✏️ Worked Examples

Q: Print the first and last letter of a word.
A: word[0] is the first letter and word[-1] is the last.

Q: Check whether a word is a palindrome.
A: word == word[::-1] is True for words like level. Reversing with [::-1] and comparing is the shortest way.

Q: Turn the sentence "i love code" into Title Case.
A: print("i love code".title()) shows I Love Code.

🚫 Common Mistakes

- Starting to count at 1. The first character is index 0.
- Expecting `s.upper()` to change `s`. Save it: `s = s.upper()`.

- Forgetting the quotes around text, or mixing quote types.
- Using `+` to add a number to text without `str()`.
- Asking for an index that does not exist, which causes an IndexError.

👨‍👧 Parent Corner

- Play a letters game: pick a word, then ask your child for the third letter, the last letter or the word backwards.
- Ask them to write a program that checks a password has at least 8 characters.
- Let them make a secret message program by reversing or changing text.
- Remind them that strings are used in every app and website.
- Ask them to explain the difference between a method and a function in their own words.

🎯 Quick Check

1. What is the index of the first character?

✅ 0.

2. What does "hello"[1:3] give?

✅ el, because the stop index 3 is not included.

3. Which method makes text uppercase?

✅ .upper().

4. How do you split a sentence into words?

✅ sentence.split().

5. What does len("cat") return?

✅ 3.